

IoC와 DI

Spring IoC 컨테이너, Bean, 의존성 주입

작성 기준: 원문 페이지를 확인한 뒤 만든 학습용 한국어 번역 요약본입니다. 공식문서 전체 문장을 그대로 복제한 전문 직역본은 아니며, 원문 URL을 함께 남겼습니다.

[1] 전체 구조 설명

- Spring IoC는 객체가 직접 의존 객체를 찾거나 만들지 않고, 컨테이너가 생성과 연결을 맡는 구조다.
- DI는 IoC의 특수한 형태로, 객체가 필요한 협력 객체를 생성자, 팩토리 메서드 인자, 프로퍼티를 통해 받는다.
- BeanFactory는 기본 설정과 객체 관리 기능을 제공하고, ApplicationContext는 AOP, 국제화 메시지, 이벤트, 웹 컨텍스트 같은 기능을 더한다.
- Spring에서 bean은 IoC 컨테이너가 만들고 조립하고 관리하는 애플리케이션 객체다.

[2] 문서별 직역체 학습 정리

IoC Container and Beans

- IoC는 제어의 방향이 바뀐다는 뜻이다. 객체가 의존성을 직접 new 하거나 Service Locator로 찾는 대신, 컨테이너가 의존 객체를 주입한다.
- Spring IoC 컨테이너의 기반은 org.springframework.beans와 org.springframework.context 패키지다.
- ApplicationContext는 BeanFactory의 확장으로, 일반 애플리케이션에서는 대부분 ApplicationContext를 기준으로 설명한다.
- Bean은 단순한 Java 객체일 수 있지만, Spring 컨테이너가 생명주기와 의존성 관계를 관리한다는 점에서 일반 객체와 구분된다.

Dependency Injection

- DI는 객체가 협력 객체를 어디서 찾을지 알 필요가 없게 한다. 그 결과 클래스는 더 느슨하게 결합되고 테스트하기 쉬워진다.
- 주요 방식은 생성자 기반 주입과 setter 기반 주입이다.
- 생성자 주입은 필수 의존성을 객체 생성 시점에 강제할 수 있어, 서비스 계층의 핵심 의존성에 적합하다.
- Setter 주입은 선택적 의존성이나 나중에 바뀔 수 있는 설정에 쓸 수 있지만, 필수 의존성을 누락할 위험이 있다.
- DI 대상 클래스는 특별한 Spring 인터페이스를 구현하지 않아도 되는 POJO일 수 있다. 이 점이 테스트 가능성을 높인다.

[3] 핵심 실행 흐름

Bean 정의 수집 → 컨테이너가 객체 생성 → 생성자/프로퍼티/팩토리 메서드로 의존성 연결 → Bean 생명주기 관리 → 애플리케이션 코드에서 주입된 객체 사용

[4] 중요한 포인트 정리

- IoC는 큰 원칙이고 DI는 그 원칙을 구현하는 대표 방식이다.
- 생성자 주입은 불변성과 필수 의존성 표현에 유리하다.

- ApplicationContext는 BeanFactory보다 실무에서 더 자주 쓰는 완성형 컨테이너다.

원문 URL

<https://docs.spring.io/spring-framework/reference/core/beans/introduction.html>

<https://docs.spring.io/spring-framework/reference/core/beans/dependencies/factory-collaborators.html>